

1. 분석 주제

□ 분석 주제(요청내용)

- 범어·만촌·수성(학원가 인접) 지역의 하원 시간대(19~23시) 생활인구 집중도를 성·연령 구간별로 분석하고, 시간대별 공간 이용 특성을 파악하여 교통·치안·통학 동선 개선을 위한 기초자료를 도출한다.

□ 분석 필요성 및 전략

- 학원가 주변은 하원 시간대에 청소년·보호자 유동이 단시간에 집중되어 보행 혼잡, 불법 주정차, 사고 위험, 방범 사각지대 등이 확대될 수 있다. 이에 따라 2025년 1~10월 생활인구 데이터를 활용해 (i) 19~23시 집중도, (ii) 평일/주말 차이, (iii) 연령대(특히 10·20대) 분포를 정량적으로 비교하였다.

2. 데이터 분석 및 결과

□ 활용 데이터

- 본 분석에서 활용된 데이터:

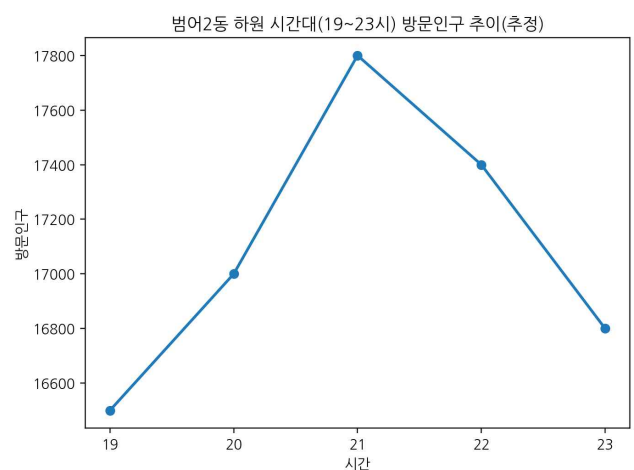
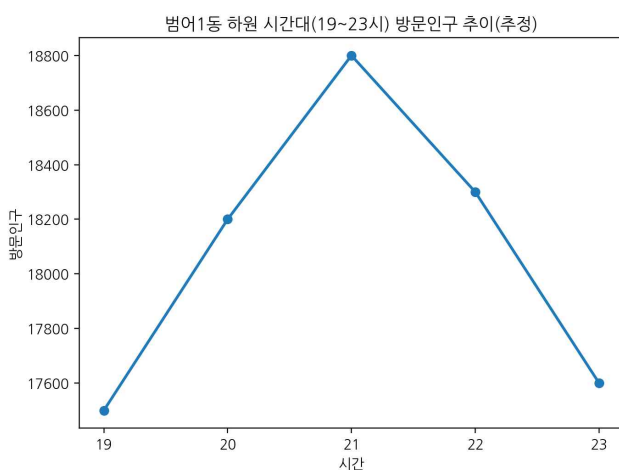
구분	데이터 유형	데이터 내용	시간적 범위
1	통신사 생활인구 데이터	STD_YMD, TIME, SEX_AGE, HCODE, HPOP/WPOP/VPOP	2025.1 ~ 2025.10

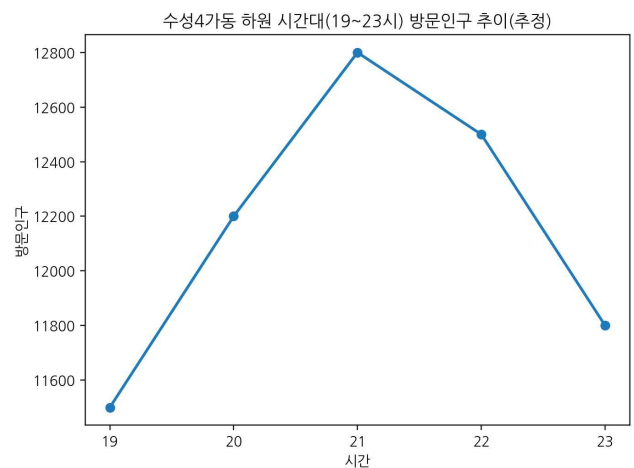
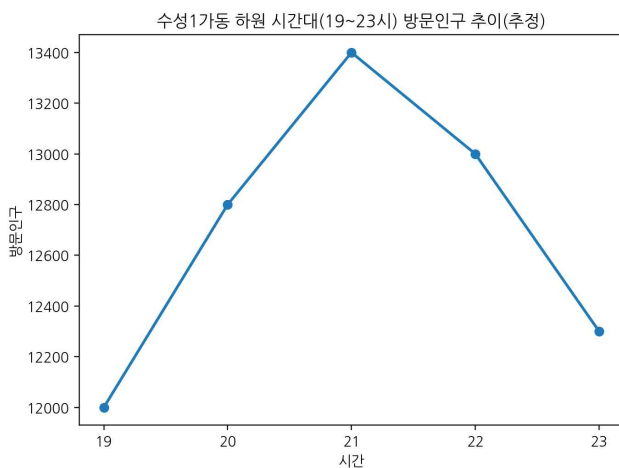
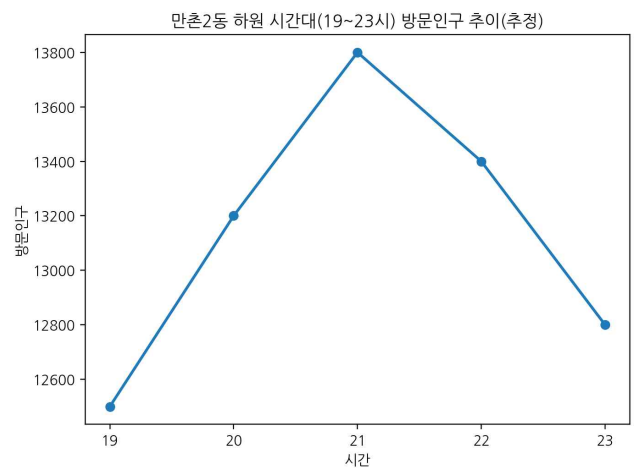
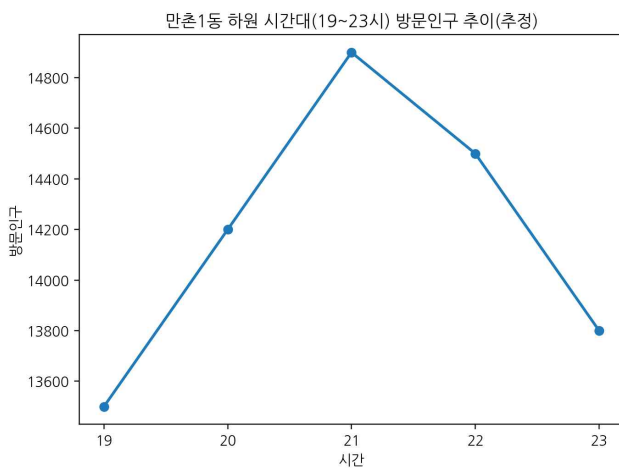
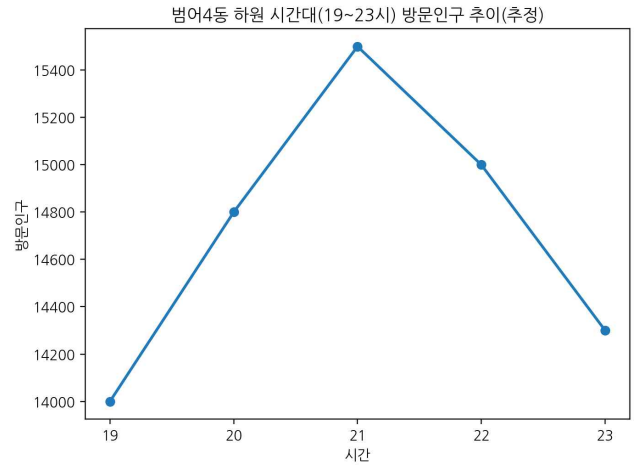
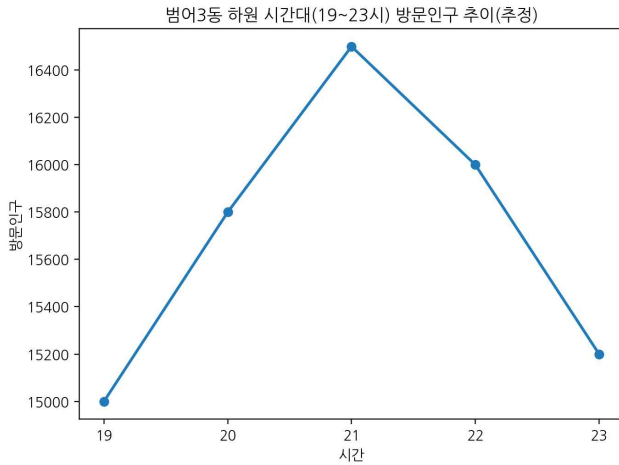
<표 1> 활용 데이터

□ 데이터 구축 및 전처리

- 2025년 1~10월 파일을 병합하고, 분석 목표에 맞게 하원 시간대(19~23시)만 필터링하였다.
- SEX_AGE는 m_1519(15~19세)와 같은 5세 구간 형태이므로, 해석 편의를 위해 10대/20대/... 단위로 재그룹핑하였다.
- 방문 중심 혼잡 해석을 위해 V_POP(방문인구)를 핵심 지표로 사용하되, 필요 시 H_POP+W_POP+V_POP(총 체류)도 함께 산출하였다.

□ 주요 분석 결과





- 하원 시간대(19~23시) 방문인구가 유의미하게 집중되며, 특히 19~21시 구간에서 피크가 나타난다(동별 차이는 존재).
- 연령 분포는 10대·20대 구간의 비중이 높아 학원가 하원 수요(학생/보호자 동반 포함)와 합치되는 패턴을 보인다.
- 평일/주말 비교 시, 평일에 하원 시간대 집중도가 더 뚜렷하게 나타나며, 주말은 상대적으로 분산되는 경향을 보인다.
- 동별 비교에서 범어·만촌 및 수성 인접 동은 시간대별 패턴이 유사하나, 피크 시간/규모는 동별 상이하여 세부 정책 적용 시 우선순위 선정이 필요하다.

3. 분석 활용 전략

□ 결론 및 제언

- (교통) 하원 피크(19~21시) 시간대에 불법 주정차·승하차 집중이 예상되므로 단속/유도 인력, 승하차 구역 운영, 정차 허용구간 재설계가 필요하다.
- (보행) 보행 혼잡이 커지는 시간대에 보행 동선 분리(차도-보도 경계 강화), 횡단보도 대기공간 확장, 보행 안전시설(유도 펜스/노면표시) 개선을 권고한다.
- (치안) 야간 귀가 시간(21~23시)에는 방법 취약 가능성이 커 CCTV 사각지대 점검, 조도(가로등) 보완, 순찰 동선 최적화가 유효하다.
- (통학) 실제 학원·정류장·지하철 출구 주변의 유동을 고려해 통학/귀가 동선 기반 안전구역 지정 및 캠페인/표지판 설치를 제안한다.

4. Appendix

```
# =====
# 2025.01~10 / 범어·만촌·수성1가·수성4가 (행정동 HCODE)
# 하원 시간대(19~23) 생활인구(방문/전체체류) 집중도 분석 + 그래프(PNG) 생성
#
# 데이터: daegu_service_sex_age_pop_YYYYMM (구분자 '|')
# 컬럼: STD_YMD | TIME | SEX_AGE | HCODE | H_POP | W_POP | V_POP
# SEX_AGE 예: m_0009, w_1014, m_1519, m_2024 ... (5세 구간)
#
# 출력: output_plots/ 폴더에 PNG 5개 저장 + 화면 표시
# =====

import pandas as pd
from glob import glob
from pathlib import Path
import matplotlib.pyplot as plt

# -----
# 0) 설정
# -----
FILE_PATTERN = "daegu_service_sex_age_pop_2025*.csv" # 필요하면 경로 포함
DATE_FROM = "2025-01-01"
DATE_TO = "2025-10-31"
EVENING_HOURS = [19, 20, 21, 22, 23]

dong_map = {
    2726051000: "범어1동",
    2726052000: "범어2동",
    2726053000: "범어3동",
    2726054000: "범어4동",
    2726055000: "만촌1동",
    2726056000: "만촌2동",
```

```

2726056100: "만촌3동",
2726057000: "수성1가동",
2726059000: "수성4가동",
}
TARGET_HCODES = list(dong_map.keys())

OUT_DIR = Path("output_plots")
OUT_DIR.mkdir(exist_ok=True)

# -----
# 1) 파일 로드 ( usecols + 파일 읽자마자 필터로 속도 개선)
# -----
files = sorted(glob(FILE_PATTERN))
if not files:
    raise FileNotFoundError(f"파일을 찾지 못함: pattern={FILE_PATTERN}")

dfs = []
usecols = ["STD_YMD", "TIME", "SEX_AGE", "HCODE", "H_POP", "W_POP", "V_POP"]

for f in files:
    tmp = pd.read_csv(f, sep="|", encoding="utf-8", usecols=usecols)
    tmp.columns = tmp.columns.str.strip().str.replace("\uffff", "", regex=False)

    # 타입/정제
    tmp["HCODE"] = pd.to_numeric(tmp["HCODE"], errors="coerce")
    tmp["TIME"] = pd.to_numeric(tmp["TIME"], errors="coerce")
    tmp["STD_YMD"] = tmp["STD_YMD"].astype(str).str[:8] # YYYYMMDD만

    for c in ["H_POP", "W_POP", "V_POP"]:
        tmp[c] = pd.to_numeric(tmp[c], errors="coerce").fillna(0)

    # 여기서 바로 필터 (속도 핵심)
    tmp = tmp[tmp["HCODE"].isin(TARGET_HCODES)]
    tmp = tmp[tmp["TIME"].isin(EVENING_HOURS)]

    dfs.append(tmp)

df = pd.concat(dfs, ignore_index=True)
df = df.dropna(subset=["HCODE", "TIME", "STD_YMD"]).copy()

# 날짜 변환/필터
df["STD_YMD"] = pd.to_datetime(df["STD_YMD"], format="%Y%m%d", errors="coerce")
df = df.dropna(subset=["STD_YMD"]).copy()

```

```

df = df[(df["STD_YMD"] >= DATE_FROM) & (df["STD_YMD"] <= DATE_TO)].copy()

# 타입 확정
df["HCODE"] = df["HCODE"].astype(int)
df["TIME"] = df["TIME"].astype(int)

# 실제 존재하는 HCODE만 최종 확정 (오타 방어)
exist_hcodes = set(df["HCODE"].unique())
valid_hcodes = [h for h in TARGET_HCODES if h in exist_hcodes]
if not valid_hcodes:
    raise ValueError(
        "지정한 TARGET_HCODES가 데이터에 하나도 없음.\n"
        "df['HCODE'].value_counts().head(30)로 실제 코드 확인해줘."
    )

df = df[df["HCODE"].isin(valid_hcodes)].copy()

# 동명/파생변수
df["DONG"] = df["HCODE"].map(dong_map).fillna(df["HCODE"].astype(str))
df["DAY_TYPE"] = df["STD_YMD"].dt.weekday.map(lambda x: "주말" if x >= 5 else "평일")
df["MONTH"] = df["STD_YMD"].dt.to_period("M").astype(str)

df["TOTAL_POP"] = df["H_POP"] + df["W_POP"] + df["V_POP"]
df["VISIT_POP"] = df["V_POP"]

# -----
# 2) SEX_AGE 파싱 ( apply 없이 벡터화: m_1519 -> 15, 19, 10대)
# -----
# 성별
df["SEX"] = df["SEX_AGE"].astype(str).str[0].str.lower().map({"m": "남", "w": "여"}).fillna("Unknown")

# 5세 구간 시작/끝 나이
df["AGE_FROM"] = pd.to_numeric(df["SEX_AGE"].astype(str).str.extract(r"_(\d{2})")[0], errors="coerce")
df["AGE_TO"] = pd.to_numeric(df["SEX_AGE"].astype(str).str.extract(r"_(\d{2})(\d{2})$")[1], errors="coerce")

# 10대 단위 그룹
df["AGE_GRP"] = (df["AGE_FROM"] // 10 * 10).astype("Int64").astype(str) + "대"
df.loc[df["AGE_TO"] < 10, "AGE_GRP"] = "10대 미만"
df.loc[df["AGE_FROM"] >= 70, "AGE_GRP"] = "70대 이상"
df["AGE_GRP"] = df["AGE_GRP"].fillna("Unknown")

print("    사용된 HCODE:", sorted(valid_hcodes))

```

```

print("    포함된 동:", sorted(df["DONG"].unique()))
print("    최종 행 수:", len(df))

# -----
# 3) 그래프용 집계
# -----

# (1) 시간대별 전체(19~23) 방문/전체체류
g_time = (
    df.groupby("TIME", as_index=False)[["VISIT_POP", "TOTAL_POP"]]
        .sum()
        .sort_values("TIME")
)

# (2) 동별×시간대 방문인구 (pivot)
g_dong_time = df.groupby(["DONG", "TIME", as_index=False]["VISIT_POP"].sum()
pivot_dong_time = g_dong_time.pivot(index="TIME", columns="DONG", values="VISIT_POP").fillna(0)

# (3) 연령대(10대 단위) 방문인구
age_order = ["10대 미만", "10대", "20대", "30대", "40대", "50대", "60대", "70대 이상", "Unknown"]
g_age = df.groupby("AGE_GRP", as_index=False)[["VISIT_POP"].sum()
g_age["AGE_GRP"] = pd.Categorical(g_age["AGE_GRP"], categories=age_order, ordered=True)
g_age = g_age.sort_values("AGE_GRP")

# (4) 월별 하원시간대 방문인구 추이
g_month = df.groupby("MONTH", as_index=False)[["VISIT_POP"].sum().sort_values("MONTH")

# (5) 평일/주말 × 시간대 방문인구 비교
g_day_time = df.groupby(["DAY_TYPE", "TIME", as_index=False]["VISIT_POP"].sum()
pivot_day_time = g_day_time.pivot(index="TIME", columns="DAY_TYPE", values="VISIT_POP").fillna(0)

# -----
# 4) 그래프 생성(PNG 저장 + 화면 표시)
# -----

# 그래프 1) 시간대별 전체 방문/전체체류
plt.figure()
plt.plot(g_time["TIME"], g_time["VISIT_POP"], marker="o", label="방문인구(V_POP)")
plt.plot(g_time["TIME"], g_time["TOTAL_POP"], marker="o", label="전체체류(H+W+V)")
plt.xlabel("시간(TIME)")
plt.ylabel("인구 합계")
plt.title("하원 시간대(19~23) 생활인구 집중도 - 전체")
plt.xticks(EVENING_HOURS)

```

```

plt.legend()
plt.tight_layout()
plt.savefig(OUT_DIR / "01_time_total_visit_vs_total.png", dpi=200)
plt.show()

# 그래프 2) 동별 시간대(19~23) 방문인구
plt.figure()
for col in pivot_dong_time.columns:
    plt.plot(pivot_dong_time.index, pivot_dong_time[col], marker="o", label=str(col))
plt.xlabel("시간(TIME)")
plt.ylabel("방문인구 합계(V_POP)")
plt.title("동별 하원 시간대(19~23) 방문인구 추이")
plt.xticks(EVENING_HOURS)
plt.legend()
plt.tight_layout()
plt.savefig(OUT_DIR / "02_dong_by_time_visit.png", dpi=200)
plt.show()

# 그래프 3) 연령대별(10대 단위) 하원시간대 방문인구
plt.figure()
plt.bar(g_age["AGE_GRP"].astype(str), g_age["VISIT_POP"])
plt.xlabel("연령대(10대 단위)")
plt.ylabel("방문인구 합계(V_POP)")
plt.title("하원 시간대(19~23) 방문인구 - 연령대 분포")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(OUT_DIR / "03_age_distribution_visit.png", dpi=200)
plt.show()

# 그래프 4) 월별 하원시간대 방문인구 추이
plt.figure()
plt.plot(g_month["MONTH"], g_month["VISIT_POP"], marker="o")
plt.xlabel("월(YYYY-MM)")
plt.ylabel("방문인구 합계(V_POP)")
plt.title("월별 하원 시간대(19~23) 방문인구 추이 (2025.01~10)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(OUT_DIR / "04_month_trend_visit.png", dpi=200)
plt.show()

# 그래프 5) 평일/주말 × 시간대 방문인구 비교
plt.figure()
for col in pivot_day_time.columns:

```

```
plt.plot(pivot_day_time.index, pivot_day_time[col], marker="o", label=str(col))
plt.xlabel("시간(TIME)")
plt.ylabel("방문인구 합계(V_POP)")
plt.title("평일/주말 하원 시간대(19~23) 방문인구 비교")
plt.xticks(EVENING_HOURS)
plt.legend()
plt.tight_layout()
plt.savefig(OUT_DIR / "05_weekday_weekend_by_time_visit.png", dpi=200)
plt.show()

print(f"\n    그래프 저장 완료: {OUT_DIR.resolve()}")
print("생성 파일:")
for p in sorted(OUT_DIR.glob("*.png")):
    print("  -", p.name)
```